Optimizing Novelty of Top-k Recommendations using Large Language Models and Reinforcement Learning

Amit Sharma
Microsoft Research
Bengaluru, India
amshar@microsoft.com

Hua Li
Microsoft Bing Ads
Redmond, USA
huli@microsoft.com

Xue Li
Microsoft Bing Ads
Mountain View, USA
xeli@microsoft.com

Jian Jiao
Microsoft Bing Ads
Redmond, USA
jiajia@microsoft.com

arXiv:2406.14169v1 [cs.IR] 20 Jun 2024

ABSTRACT

Given an input query, a recommendation model is trained using user feedback data (e.g., click data) to output a ranked list of items. In real-world systems, besides accuracy, an important consideration for a new model is novelty of its top-k recommendations w.r.t. an existing deployed model. However, novelty of top-k items is a difficult goal to optimize a model for, since it involves a non-differentiable sorting operation on the model's predictions. Moreover, novel items, by definition, do not have any user feedback data. Given the semantic capabilities of large language models, we address these problems using a reinforcement learning (RL) formulation where large language models provide feedback for the novel items. However, given millions of candidate items, the sample complexity of a standard RL algorithm can be prohibitively high. To reduce sample complexity, we reduce the top-k list reward to a set of item-wise rewards and reformulate the state space to consist of ⟨query, item⟩ tuples such that the action space is reduced to a binary decision; and show that this reformulation results in a significantly lower complexity when the number of items is large. We evaluate the proposed algorithm on improving novelty for a query-ad recommendation task on a large-scale search engine. Compared to supervised finetuning on recent <query, ad>pairs, the proposed RL-based algorithm leads to significant novelty gains with minimal loss in recall. We obtain similar results on the ORCAS query-webpage matching dataset and a product recommendation dataset based on Amazon reviews.

CCS CONCEPTS

• **Information systems** → **Novelty in information retrieval**; **Online advertising**.

KEYWORDS

Recommendation System, Novelty, Large Language Models, Reinforcement Learning

ACM Reference Format:
Amit Sharma, Hua Li, Xue Li, and Jian Jiao. 2024. Optimizing Novelty of Top-k Recommendations using Large Language Models and Reinforcement Learning . In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), August 25–29, 2024, Barcelona, Spain.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3637528.3671618

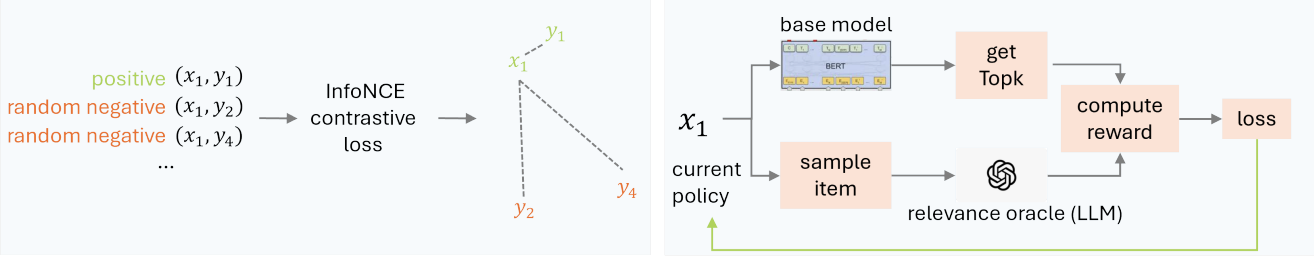
KDD '24, August 25–29, 2024, Barcelona, Spain
© 2024 Copyright held by the owner/author(s).
This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), August 25–29, 2024, Barcelona, Spain*, https://doi.org/10.1145/3637528.3671618.

1 INTRODUCTION

Given a user's profile or an input query, the recommendation problem is to fetch a ranked list of top-k items based on a task-specific goal. We consider the retrieval layer of a recommendation system, where the input is typically millions of candidate items and output is hundreds of ranked items (e.g., k=200). For the retrieval layer, semantic relevance is a common goal and models are trained to rank relevant items higher [9]. A common way to train such recommendation models is to use supervised learning on user feedback data such as clicks, using losses such as contrastive learning that encourage representations of clicked query-item pairs to be closer to each other than negative (or random) query-item pairs [8].

However, in addition to relevance, real-world recommendation systems typically have additional goals to optimize for the predicted top-k items. For instance, an important goal is that a candidate model predicts top-k items that are *novel* w.r.t. the existing deployed models in the system [15]. Novelty is a desirable property since it can avoid showing repetitive or redundant items to a user, enhance global coverage over all available items, and help avoid any systematic bias towards previously clicked items in the system [11]. However, it is difficult to train a model to optimize novelty of top-k items since evaluating novelty requires a non-differentiable sorting operation to obtain the top-k list. Moreover, novel items, by definition, do not have any user feedback data corresponding to the particular query. As a result, getting relevance feedback on novel items requires conducting an online experiment showing exploratory, novel items to the user, which can be a costly and infeasible procedure for many systems.

To obtain scalable feedback for novel items, we utilize recent results that show that large language models (LLMs) like GPT-3.5 or GPT-4 can match or surpass quality of crowd-sourced relevance feedback for recommendation tasks, such as matching user input to ad keywords [10] or matching user history to suggested movies and games [12]. Specifically, we propose that LLMs can be used as *reward models* to provide relevance feedback for novel items, using an appropriate task-specific prompt. As a result, (noisy) relevance feedback for novel items can be obtained at scale and be used for



(a) Supervised learning (b) PG-Ret: Proposed reinforcement learning approach

Figure 1: Our goal is to optimize retrieval models for novelty. *Left:* Supervised learning with a contrastive loss is unable to optimize novelty directly since it involves a non-differentiable top-k sorting operation. *Right:* the proposed RL approach with explicit optimization for the novelty of top-k retrieved items (w.r.t. to a base model) using LLM-based relevance feedback.

both offline model training and evaluation, with human feedback restricted to only evaluating the final model in an A/B test.

For handling non-differentiability of the novelty metric, we consider a reinforcement learning (RL) formulation, as in past work on optimizing metrics such as diversity [20] and popularity [32, 33] of top-k predicted items. Specifically, the retrieval problem can be framed as a one-step RL problem where the input query is the state, the top-k predicted items constitute the action, and their relevance and novelty constitute the environment reward. However, the large action space in retrieval tasks—equal to the number of available items that can be predicted for an input query (typically millions)—presents a challenge due to the data requirements for RL algorithms. While we use a policy gradient RL algorithm that does not explicitly parameterize the action space and hence is better suited for large action spaces, the finite sample convergence results of policy gradient algorithms [23] show a direct dependence of policy error on the square of the action space size. Consequently, the resultant algorithms are reported to not work well beyond tens of items [22].

To address the issue of optimizing over large action spaces, we make two contributions. First, rather than the standard practice of training RL algorithms from scratch for recommendation systems [16, 32, 33, 38], we argue that it is more practical to consider RL as a tool to *finetune* an existing supervised learning model that has been trained on user feedback data. Such an approach has led to substantial gains in training image [28] and language [26] models and we extend it for recommendation systems. We also provide a formal justification for this line of work: assuming that the existing model has a higher probability of selecting the optimal action (item) for a query than a uniformly random policy, we show that initializing from an existing model reduces the sample complexity requirements for a policy gradient algorithm.

Second, we propose a reformulation of the problem that further reduces the sample complexity. Rather than considering a query as the state, we consider the (query, item) pair as a state. As a result, the action space is converted to a binary space: whether the item is preferred by the task-specific goal (e.g., relevance and novelty) or not. Theoretically, our formulation reduces the dependence of convergence error from quadratic to linear in the action space, which is a substantial reduction whenever the action space is large. Crucially, even though the action space is binary given a query-item pair, policy optimization can use rewards based on the top-k retrieved list from the current policy, which was not possible in supervised learning. At the same time, the reduction leads to an empirical advantage: we can develop a scalable, batch-wise training algorithm for optimizing the policy using loss functions that are well-suited for large-scale recommendation, such as those based on contrastive learning. We call the resultant algorithm, *PG-Ret*, Policy Gradient for large-scale Retrieval.

We evaluate PG-Ret on three datasets: a proprietary dataset for query-ad recommendation task, a public dataset for query-webpage matching task, and a product recommendation dataset based on Amazon reviews. In the first dataset, the task is produce top-k items that are novel compared to a given production model on a commercial search engine. The training data consists of recently collected <query, ad>pairs from the search engine's logs and has 33M actions. We find that PG-Ret leads to significant improvement (2X-5X) in novelty of top-k retrieved items compared to a supervised model trained on the same training data, while incurring minimal drop in precision and recall. Online A/B test shows a statistically significant gain of 1% in retrieval density, the average number of relevant ad keywords fetched per query. The second dataset is a public dataset, ORCAS, for matching queries to webpage titles. We obtain a similar result: PG-Ret leads to over 2X improvement in novelty for the top-20 items while incurring a small drop in recall and precision. Finally, PG-Ret can also be applied to general recommendation scenarios beyond query-based tasks. We apply it to an Amazon user-product recommendation task and find significant gains in novelty compared to supervised finetuning.

To summarize, we propose a technique based on the policy gradient algorithm that is simple to implement (see Algorithm 1) and shows the benefit of finetuning existing models with RL for task-specific goals such as novelty. Our contributions include,

- A simple, model-agnostic algorithm to finetune retrieval models that works for large action spaces (millions of actions) and arbitrary task-specific goals.
- Leveraging relevance feedback from LLMs; thus allowing for optimization of novel items directly since novel items, by definition, do not have user feedback data.

- Theoretical and empirical justification of the method for increasing novelty in recommendation systems, including evaluation of the method for predicting ad keywords on a commercial search engine.

2 RELATED WORK

2.1 Enhancing novelty of top-k results.

Outputting novel items compared to existing algorithms is an important goal for recommendation systems [31]. A common approach is to consider a multi-objective optimization problem [29] with relevance and novelty as the two objectives [6, 14]. Recently, reinforcement learning has been used for novelty optimization since RL can be applied to non-differentiable objectives like novelty [16, 33, 38]. However, past work uses approximations of novelty, such as an item being less popular [32–34] or belonging to less popular categories [38], to enhance novelty, since truly novel items—those that have not been shown by the current system—would not have any user feedback data for training. To ascertain relevance of truly novel items, heuristics like estimating the curiosity of a user (more curiosity admits more novel items as relevant) have been proposed [40], but no systematic way to estimate relevance exists. Consequently, without relevance feedback, there exists a direct tradeoff between optimizing novelty and relevance of a top-k list. In this work, we use the semantic capabilities of LLMs to propose a general, scalable method to estimate relevance of novel items. As a result, we can directly optimize for the novelty objective without sacrificing relevance.

Note that encouraging novelty in top-k items is different from addressing the cold-start problem [30, 37]. Novel items are defined wrt. a query whereas cold-start items are defined globally for a system. In many cases, a novel item wrt. a query may be a popular item globally. As a result, methods for the cold-start problem may not directly apply to novelty optimization.

2.2 RL with large action spaces.

Given a query, information retrieval can be divided into two stages [13]; 1) retrieval of relevant items from a large candidate pool of items; 2) ranking the retrieved items to select a smaller set of items that are shown to the user. RL algorithms find many applications in the ranking phase, including contextual bandits [19], markov decision processes [35], and policy gradient algorithms [27, 39]. Many of these algorithms introduce a task-specific loss to the ranking problem. However, applying these techniques to the retrieval phase is challenging because of the large number of candidate items (actions).

Large action spaces is a general challenge for RL beyond retrieval models [7], even when using policy gradient, a method that does not explicitly parameterize the action space and thus is better suited for higher number of actions. For sequential recommendation problems, appropriately parameterizing the policy and using off-policy correction helps in scaling to large actions [2]. For one-step RL problems, recent theoretical work in contextual bandits [21, 41] tries to address the large actions problem. However, the focus of these approaches is on obtaining an optimal solution from scratch, which may be difficult in practice and misses the complementary benefits of supervised learning over user feedback. Instead, we finetune an existing supervised model, as done by [28] for computer vision models.

2.3 LLMs for information retrieval.

Recently, LLMs like GPT-3.5 have been applied to retrieval tasks in a zero-shot setting with encouraging results [1, 5, 12]. Instead of using compute-intensive LLMs directly for the retrieval task, here we aim to use LLMs as reward models to train an efficient, small retrieval model.

3 OPTIMIZING NOVELTY W/ LLM FEEDBACK

3.1 Problem statement: Novelty optimization

Commonly used retrieval models use a bi-encoder architecture [8], where the same neural network model embeds a query and item into a common representation space. The top-k items are selected based on the nearest neighbors to a query, as measured by a suitable distance function over the embeddings (e.g., cosine similarity). The encoder ϕ is typically optimized using a variant of contrastive learning, encouraging that positive <query,item> pairs in the training set should be closer in embedding space than non-positive pairs. Non-positive pairs may be random pairs [8] or mined from train data[4]. Thus, given a set of queries X and items Z, a train dataset $D \sim \mathcal{D}$ with positive query-item pairs, and a similarity function $\texttt{sim}$, the trained retriever $\hat{\phi}$ can be written as (using the InfoNCE contrastive loss function [25]),

$$\hat{\phi} = \arg\min_{\phi} -\log \sum_{(x,z)\in\mathcal{D}} \frac{\exp(\texttt{sim}(\phi(x), \phi(z)))}{\sum_{z'\in neg(x)} \exp(\texttt{sim}(\phi(x), \phi(z')))}$$

$$\mathbf{y}(x) = \texttt{Topk}_{z\in Z}\texttt{sim}(\hat{\phi}(x), \hat{\phi}(z)) \tag{1}$$

where $\mathbf{y} = [y_1, y_2, \cdots, y_k]$ refers to the sequence of top-k items returned by a trained encoder $\hat{\phi}$ and $\texttt{Topk}$ is a non-differentiable ranking operation. At inference time, given a query, the trained encoder $\hat{\phi}$ is used to compute the similarity with each item and the top-k closest items are returned as the prediction.

Given the trained encoder $\hat{\phi}$, our goal is to finetune the encoder such that the novelty of its top-k items compared to a base model is optimized, while ensuring that the accuracy of the encoder does not decrease substantially. Specifically, we assume that there is an existing base retrieval model ψ deployed in the production recommendation system and we want to deploy another retrieval model that introduces novel items. Both the base model and the new, finetuned model would run in parallel and their results can be merged for the downstream ranking layer. Formally, novelty@k is defined as,

Definition 1. **Novelty:** Given a query x, Novelty@k (x, ϕ, ψ, L) is defined as the number of items in top-k predictions of a candidate model ϕ that do not exist in top-L predictions of the base model ψ.

Typically, L is set as the number of items that the retrieval layer sends downstream to the ranking layer (e.g., $L = 200$ in our experiments). If a candidate model outputs an item that is beyond the top-L of the base model, then it will result in a new item added to the recommendation system's ranking layer. Note that k need not be the same as L. For efficiency reasons, we may be interested in